

47. Deux boîtes qui se touchent

Deux lutins se touchent si leurs carrés se chevauchent : quatre comparaisons, pas une de plus.

1. Ce qu'il faut comprendre

Ramasser une pièce, se faire toucher par un ennemi, recevoir une flèche : c'est toujours la même question. **Deux lutins se touchent-ils ?** Au pixel près, on ne peut plus demander « sont-ils dans la même case ? » : il faut comparer des **boîtes**.

Chaque lutin occupe un carré de 8×8 pixels. Le héros va de x à $x + 7$ en largeur, la pièce de px à $px + 7$. Ils se chevauchent en largeur **si aucun des deux n'est entièrement à côté de l'autre** : le héros commence avant la fin de la pièce ($x < px + 8$) **et** la pièce commence avant la fin du héros ($px < x + 8$).

Exemple chiffré : héros en $x = 60$, pièce en $px = 66$. $60 < 74$: oui. $66 < 68$: oui. Ils se chevauchent (sur 2 pixels, de 66 à 67). Héros en $x = 50$: $66 < 58$? non — la pièce commence après la fin du héros. Pas de contact.

Ce qui est nouveau ici : `touche()`, **quatre comparaisons liées par `&&`**. Deux pour la largeur, deux pour la hauteur, avec les mêmes règles sur `y` et `py`. Les quatre doivent être vraies : si une seule est fausse, les boîtes sont séparées par au moins une ligne ou une colonne de pixels.

Quand ça touche, `score` augmente et la pièce **saute** à la place suivante d'une table gravée (chapitre 4). `(i + 1) % 3` fait tourner `i` sur 0, 1, 2, 0, 1... : le modulo du chapitre 1.

À toi : rétrécis la boîte de la pièce (compare avec `px + 2` et `px + 6`) : il faut alors vraiment marcher dessus. Beaucoup de jeux donnent aux objets une boîte plus petite que leur dessin, pour être justes avec le joueur.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **929 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	71 o	← la console, à l'allumage — la mise en route de la console
touche()	86 o	← main() — ta fonction
main()	209 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d'une division
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	272 o	← CopierTuiles — 15 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins
"SCORE"	5 o	← texte() — un texte gravé

```

PX_0          3 o ← sprite()  - une table gravée
PY_1          3 o ← sprite()  - une table gravée
Lettres gravées : CEORS0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Multiplier,
DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes
Fonctions du langage appelées :
  sprite() × 2    62 o sur place, dans main() + PX_0 (3 o), PY_1 (3 o)
  nombre() × 2    20 o sur place, dans main() + EcrireNombre (33 o)
  bouton() × 4    20 o sur place, dans main() + LireManette (37 o)
  texte()  × 1    11 o sur place, dans main() + EcrireTexte (10 o)
  image()  × 1     6 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
  touche() × 1     3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros qu'on déplace avec les 4 flèches ramasse une pièce.
// Quand il la touche, le score monte et la pièce saute à une autre place.
//
// Ce qui est nouveau : la collision de deux BOÎTES.
//  Chaque lutin est un carré de 8 × 8 pixels : le héros va de x à x + 7.
//  Deux carrés se touchent en largeur si CHACUN commence avant la fin de l'autre :
//    x < px + 8  ET  px < x + 8
//  Pareil en hauteur avec y. Il faut les 4 conditions vraies.
//  Exemple : héros x = 60, pièce px = 66 : 60 < 74 oui, 66 < 68 oui → contact.
//    héros x = 50 : 66 < 58 ? non → pas de contact.

#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près

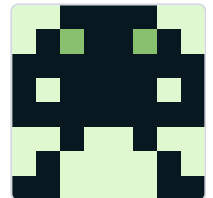
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {
  "..####..",
  ".#-##-#.",
  "#####",
  "#.####.#",
  "#####",
  "..#..#..",
  ".#....#.",
  "##....##",
};

```



à l'écran

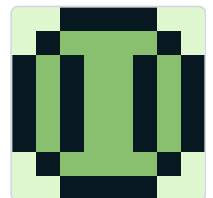
3. UN DESSIN — PIECE

```

Tuile PIECE = {
  "..####..",
  ".#---#.",
  "#-##-#.",
  "#-##-#.",
  "#-##-#.",
  "#-##-#.",
  ".#---#.",
  "..####..",
};

// Les trois places de la pièce, l'une après l'autre (en pixels).
//    place 0  1  2

```



à l'écran

4. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PX

```
const uint8_t PX[] = { 120, 24, 80 }; // x de chaque place
```

5. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PY

```
const uint8_t PY[] = { 72, 104, 40 }; // y de chaque place
```

6. UNE VARIABLE — X

```
uint8_t x = 40; // le héros, en pixels
```

7. UNE VARIABLE — Y

```
uint8_t y = 72;
```

8. UNE VARIABLE — I

```
uint8_t i = 0; // la place de la pièce en ce moment : 0, 1 ou 2
```

9. UNE VARIABLE — SCORE

```
uint8_t score = 0;

// 1 si la boîte du héros chevauche celle de la pièce, 0 sinon.
```

10. UNE FONCTION — TOUCHE()

```
uint8_t touche() {
    return x < PX[i] + 8 && PX[i] < x + 8 // en largeur
           && y < PY[i] + 8 && PY[i] < y + 8; // et en hauteur
}
```

11. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    texte(1, 1, "SCORE");
    nombre(7, 1, score);

    while (true) { // La boucle du jeu :
        image();

        if (bouton(DROITE) && x < 152) x++; // les 4 flèches, sans sortir de l'écran
        if (bouton(GAUCHE) && x > 0) x--;
        if (bouton(BAS) && y < 136) y++;
        if (bouton(HAUT) && y > 16) y--; // pas plus haut que 16 : la ligne du score

        if (touche()) { // Contact ?
            score++;
            i = (i + 1) % 3; // la pièce saute à la place suivante : 0 → 1 → 2 → 0 ...
            nombre(7, 1, score); // on n'écrit le score QUE quand il change
        }

        sprite(0, x, y, HEROS); // lutin 0 : le héros
        sprite(1, PX[i], PY[i], PIECE); // lutin 1 : la pièce, à sa place actuelle
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un héros et une pièce sur la même ligne ; en marchant dessus, le score monte et la pièce saute ailleurs.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, le score est 0
- ✓ en marchant sur la pièce, le score monte
- ✓ le contact arrive quand les boîtes se chevauchent : $x = 113$ ($x = 113$)
- ✓ la pièce a sauté à sa deuxième place

← [46. Regarder à gauche ou à droite : le miroir](#)

[sommaire](#)

[48. Lire la case devant soi : le mur](#) →